Padrão de Projeto

Memento

Preservando o Estado sem Quebrar o Encapsulamento

Também Conhecido Como

Lembrança, Retrato, Snapshot

Padrões de Projeto Delineados

Padrões de Projeto: Soluções reutilizáveis de software orientado a objetos

Disciplina

Programação Orientada a Objetos

Tema

Padrões de Projeto

Alunos

Matheus M. Tártari

Felipe M. Santos

Sumário da Apresentação

3 Definição: O Padrão Memento

3 Aplicações Típicas

4 Propósito: A Necessidade de Reversão de Estado

5 Estrutura (Elementos)

6 Fluxo de Colaboração Entre Elementos

7 - 10 Exemplo em Codigo

11 Vantagens

12 Desvantagens

13 Conclusão

14 Referências

Definição: O Padrão Memento

O Memento é um **padrão de projeto comportamental** que permite salvar e restaurar o estado anterior de um objeto sem revelar os detalhes de sua implementação.

Aplicações Típicas

- **Funcionalidade "Desfazer" (Undo/Redo):** Essencial em editores de texto, softwares gráficos e sistemas de CAD.
- **Ponto de Restauração (Checkpoints):** Utilizado em jogos ou sistemas transacionais para retornar a um estado seguro após falhas ou erros.
- **Histórico de Transações:** Em sistemas financeiros, permite a reversão de operações.

Propósito: A Necessidade de Reversão de Estado

O Memento resolve o desafio de capturar e restaurar o estado interno de um objeto (**Originator**) sem expor seus detalhes internos, garantindo o **encapsulamento**.

Problema Sem o Padrão

- Se o estado for exposto (campos públicos), o encapsulamento é violado, tornando o código frágil a mudanças.
- Se o estado for privado, uma classe externa (o gerenciador de histórico) não consegue acessá-lo para salvar ou restaurar.

Estrutura (Elementos)

Originator (Originador)

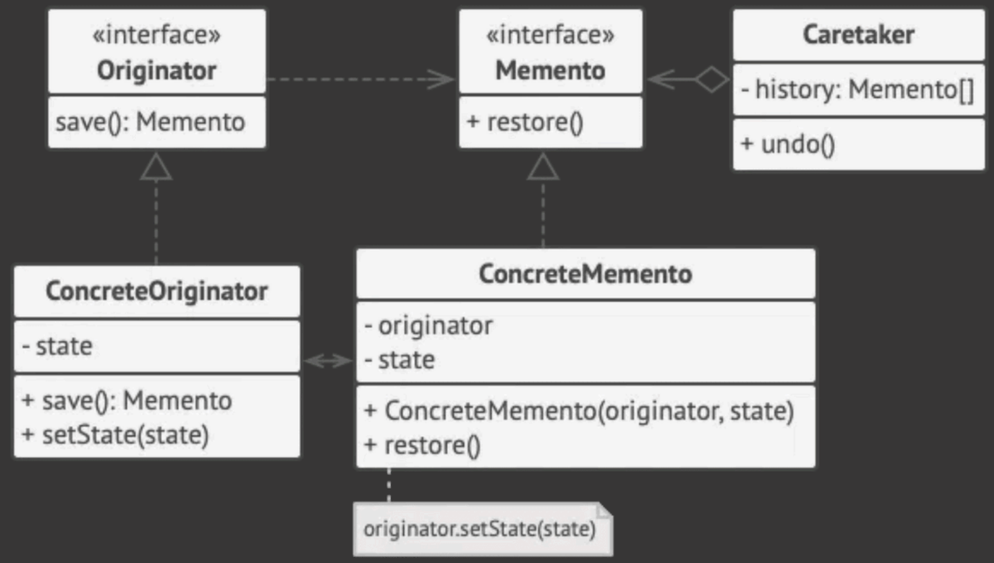
O objeto cujo estado deve ser salvo. Cria o Memento e usa o Memento para restaurar seu estado.

Memento (Lembrança/Retrato)

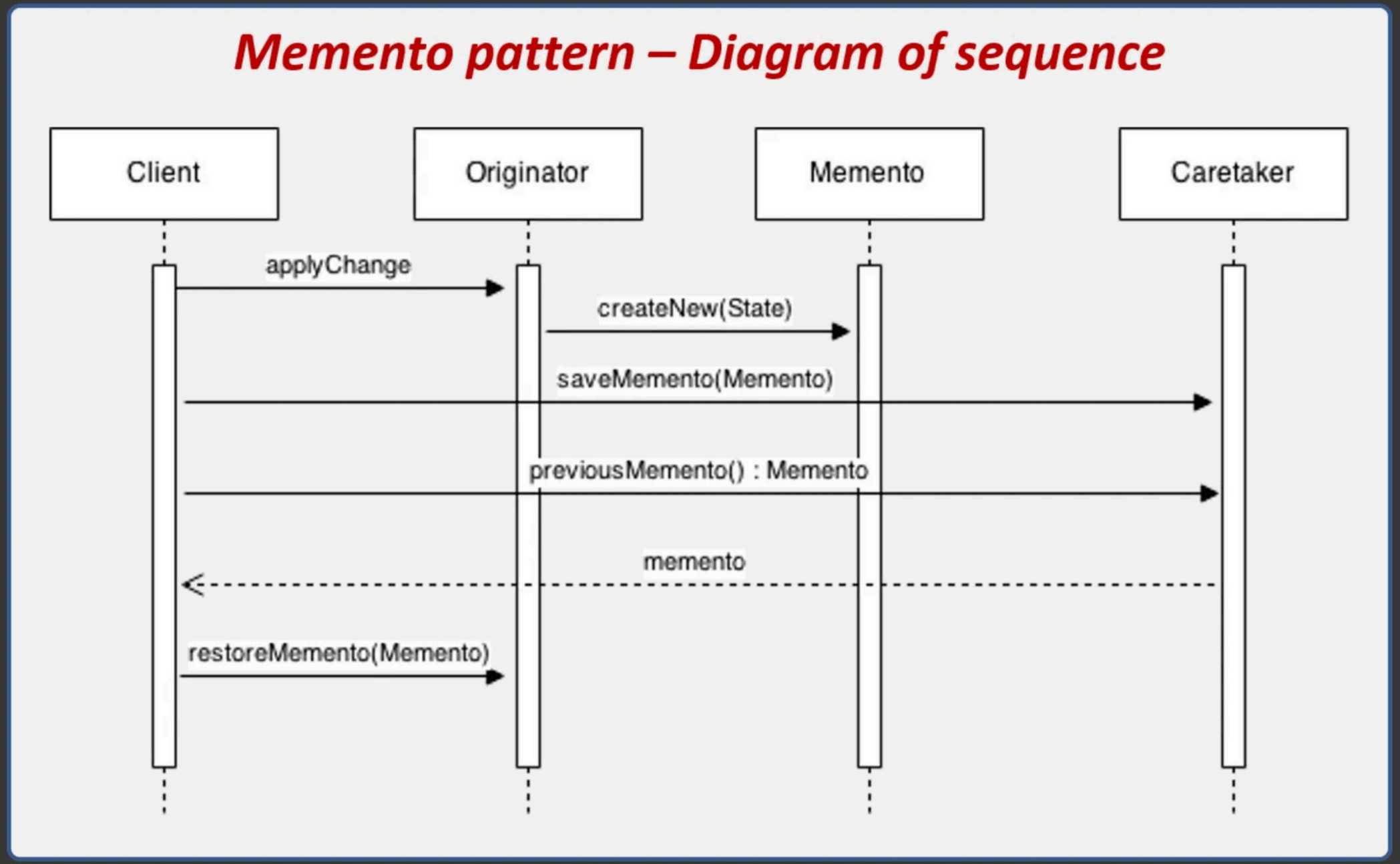
Um objeto que armazena o estado interno do Originator. É imutável e possui uma interface restrita para o Caretaker.

Caretaker (Cuidador)

Responsável por armazenar e gerenciar os Mementos (o histórico). Nunca inspeciona ou modifica o conteúdo de um Memento.



Fluxo de Colaboração Entre Elementos



Código Sem Memento: Quebra de Encapsulamento

```
class Editor {
    private String content = "";

    public void type(String words) { this.content += " " + words; }


    public String getContent() { return this.content; }

    public void setContent(String content) {
        this.content = content;
    }
}

class HistorySemMemento {
    // Guarda o estado diretamente, não um objeto Memento
    private Stack<String> history = new Stack<>();

    // O History tem de "puxar" o estado do editor
    public void save(EditorSemMemento editor) {
        history.push(editor.getContent());
    }

    public void undo(EditorSemMemento editor) {
        if (!history.isEmpty()) {
            editor.setContent(history.pop()); // Usa o "setter" perigoso
        }
    }
}

// --- Demonstração ---
public void demoSemMemento() {
```

```java
class HistorySemMemento {
    // Guarda o estado diretamente, não um objeto Memento
    private Stack<String> history = new Stack<>();

    // O History tem de "puxar" o estado do editor
    public void save(EditorSemMemento editor) {
        history.push(editor.getContent());
    }

    public void undo(EditorSemMemento editor) {
        if (!history.isEmpty()) {
            editor.setContent(history.pop()); // Usa o "setter" perigoso
        }
    }
}

// --- Demonstração ---
public void demoSemMemento() {
    EditorSemMemento editor = new EditorSemMemento();
    HistorySemMemento history = new HistorySemMemento();

    editor.type("Olá");
    history.save(editor);
    editor.type("Mundo");


    history.undo(editor);


}
```

Código Com Memento: Encapsulamento Preservado

```java
class EditorMemento {
    private final String content;

    public EditorMemento(String content) {
        this.content = content;
    }

    // Apenas o Originador (Editor) pode aceder isto
    protected String getContent() {
        return content;
    }
}

class Editor {
    private String content = "";

    public void type(String words) { this.content += " " + words; }
    public String getContent() { return this.content; }

    // Cria o Memento
    public EditorMemento save() {
        return new EditorMemento(this.content);
    }

    // Restaura a partir do Memento
    public void restore(EditorMemento memento) {
        this.content = memento.getContent();
    }
}
```

```
        return new EditorMemento(this.content);
    }

    // Restaura a partir do Memento
    public void restore(EditorMemento memento) {
        this.content = memento.getContent();
    }
}

class History {
    private Stack<EditorMemento> history = new Stack<>();

    public void push(EditorMemento memento) { history.push(memento); }
    public EditorMemento undo() { return history.pop(); }
}


public void demoComMemento() {
    Editor editor = new Editor();
    History history = new History();

    editor.type("Olá");
    history.push(editor.save());
    editor.type("Mundo");


    editor.restore(history.undo());


}
```

Análise Crítica: Vantagens e Desvantagens

Vantagens ✓

Preserva o Encapsulamento

O Caretaker não precisa conhecer os detalhes internos do Originator.

Simplifica o Caretaker

O Caretaker se torna um simples gerenciador de histórico (pilha), desacoplado da lógica de estado.

Suporta Múltiplos Desfazeres

Facilmente implementável com uma estrutura de pilha para o histórico.

Flexibilidade

Permite que o Originator decida quais partes do seu estado devem ser salvas.

Análise Crítica: Vantagens e Desvantagens

Desvantagens ✕

Custo de Memória

A criação frequente de Mementos pode consumir muita memória, especialmente com estados grandes.

Simplifica o Caretaker

O processo de criação e restauração do estado pode ser custoso em termos de processamento.

Custo de Tempo

O Caretaker deve gerenciar o ciclo de vida dos Mementos, descartando os obsoletos para liberar memória.

Linguagens Dinâmicas

Em Python, a garantia de que o Caretaker não acessará o estado interno é por convenção, não por restrição.

Conclusão: Relevância do Padrão Memento

Síntese dos Pontos Principais

1

Encapsulamento

Permite a funcionalidade de "desfazer" sem expor os dados privados do objeto.

2

Desacoplamento

Separa a responsabilidade de salvar o estado (Originator) da responsabilidade de gerenciar o histórico (Caretaker).

3

Reutilização

O Caretaker pode ser reutilizado para gerenciar o histórico de diferentes tipos de Originators.

Quando Utilizar (Ou não) o Padrão Memento:

Funcionalidade de Desfazer/Refazer: Em editores de texto, softwares gráficos, sistemas de CAD e qualquer aplicação que necessite reverter operações.

Checkpoints em Jogos: Para salvar e restaurar estados do jogo em momentos críticos, permitindo que o jogador retorne a um ponto seguro.

Estados Muito Grandes: Quando o estado do objeto é muito pesado, o armazenamento dos Mementos pode consumir muita memória.

Mudanças de Estado Muito Frequentes: Se o estado muda o tempo todo, criar Mementos constantemente torna o sistema lento e ineficiente.

Referências

Livro Base

GAMMA, Erich; **HELM, Richard**; **JOHNSON, Ralph**; **VLISSIDES, John**. *Padrões de Projeto: Soluções reutilizáveis de software orientado a objetos*. Tradução de **Marcos Marques Zambon**. Porto Alegre: **Bookman**, 2000. 364 p. (Original publicado em 1995 por Addison-Wesley).

Fontes Complementares

REFACTORING.GURU. *Memento*. Disponível em: <https://refactoring.guru/pt-br/design-patterns/memento>. Acesso em: 3 nov. 2025.

TUTORIALS POINT. *Design Patterns - Memento Pattern*. Disponível em: <https://www.tutorialspoint.com/design_pattern/memento_pattern.htm>. Acesso em: 3 nov. 2025.

WIKIPEDIA. *Memento (Informática)*. Disponível em: <https://pt.wikipedia.org/wiki/Memento_(inform%C3%A1tica)>. Acesso em: 3 nov. 2025.

DIO - DIGITAL INNOVATION ONE. *SW Design Pattern - Memento (Lembrança)*. Disponível em: <https://www.dio.me/articles/sw-design-pattern-memento-lembranca>. Acesso em: 3 nov. 2025.

Perguntas e Discussão

Dúvidas, comentários ou sugestões sobre o padrão Memento?

Espaço para Discussão

Obrigado pela atenção!

Padrão de Projeto

Memento: Preservando o Estado sem Quebrar o Encapsulamento

Gamma et al., 1995 | Padrões de Projeto: Soluções Reutilizáveis de Software Orientado a Objetos